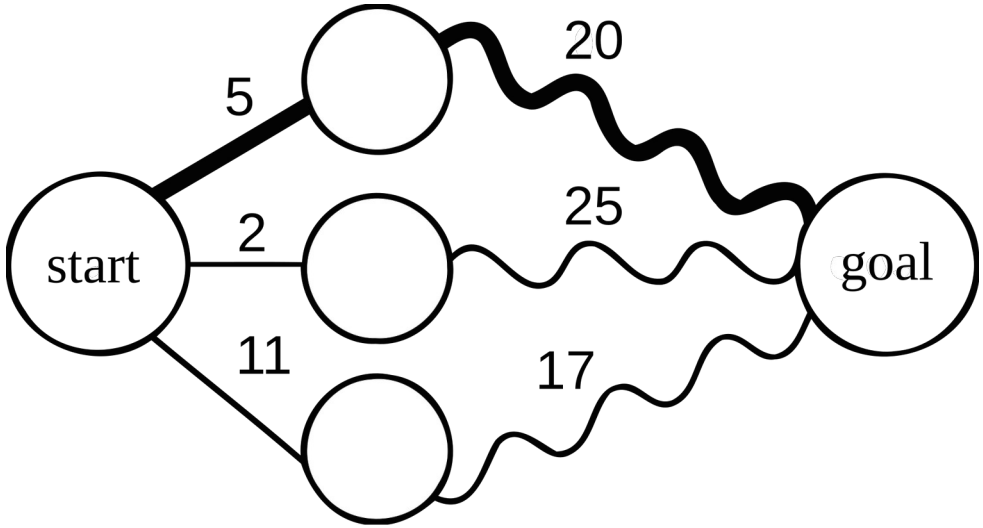
Lecture 15

Dynamic Programming I



Last time

- DAGs
- Topological sorting on directed graphs
- Recursion

Today

- Dynamic Programming Introduction

Algorithmic Paradigms

Greedy

- Make local choices
- Not all problems have an efficient greedy algorithm

Algorithmic Paradigms

Greedy

- Make local choices
- Not all problems have an efficient greedy algorithm

Dynamic Programming

- Recursively break problem into subproblems
- Employ top-down memoization or bottom-up induction to solve efficiently

Minimum Change Problem

Problem: You want to give `65` cents in as few coins as possible using the usual USD coins: `25, 10, 5, 1`

Minimum Change Problem

Problem: You want to give `65` cents in as few coins as possible using the usual USD coins: `25, 10, 5, 1`

- Solve using Greedy algorithm (Midterm exam); `O(n)` running time

Minimum Change Problem

Problem: You want to give `65` cents in as few coins as possible using the usual USD coins: `25, 10, 5, 1`

- Solve using Greedy algorithm (Midterm exam); `O(n)` running time

Problem: You want to give `6` cents in as few coins as possible using coins worth the following amounts: `4, 3, 1`

Minimum Change Problem

Problem: You want to give `65` cents in as few coins as possible using the usual USD coins: `25, 10, 5, 1`

- Solve using Greedy algorithm (Midterm exam); `O(n)` running time

Problem: You want to give `6` cents in as few coins as possible using coins worth the following amounts: `4, 3, 1`

- Greedy fails!
- Optimal change is {3, 3}

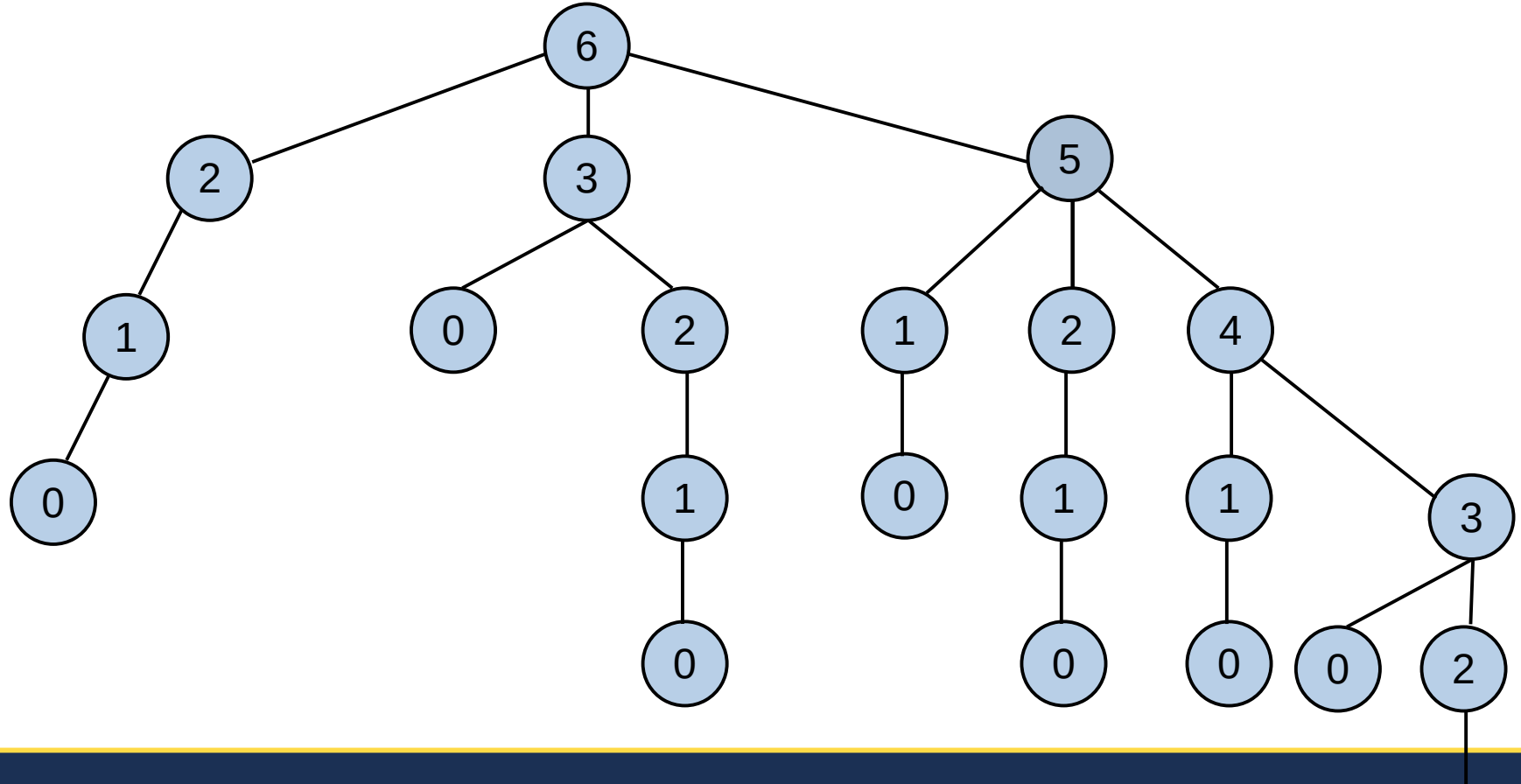
Recursive Formulation

Let A be an optimal way to give 6 cents in as few coins as possible

- Either 4, 3, or 1 belongs to A
- If 4 does, then consider the optimal way to give 6 - 4=2 cents
- If 3 does, then consider the optimal way to give 6 - 3=3 cents
- If 1 does, then consider the optimal way to give 6 - 1=5 cents

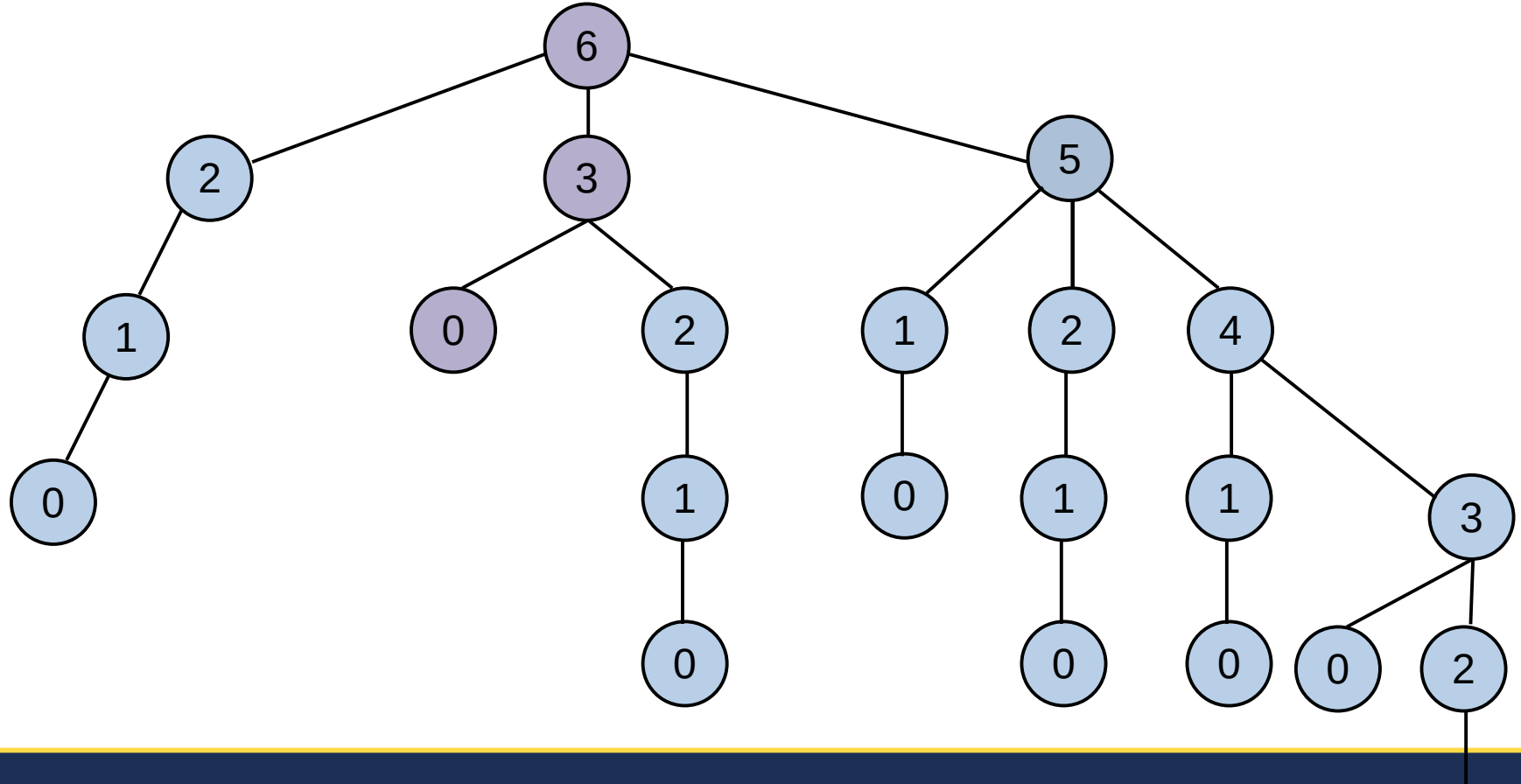
Recursion Tree

The optimal solution is the path from 6 to 0 with the shortest length



Recursion Tree

The optimal solution is the path from 6 to 0 with the shortest length



Dynamic Programming Recipe

- Step 1: Devise simple recursive algorithm for the value of the optimal solution
- Step 2: Write recurrence relation for optimal value
- Step 3: Design bottom-up iterative algorithm
- Step 4: Recover optimal solution

General Minimum Change Problem

Problem: You want to give M cents in as few coins as possible using coins worth the following amounts: $C = c_1, c_2, \ldots, c_k$. Assume you can give exact change.

- In general, greedy approach won't work (see prior example)
- Solving via recursive formulation is $O(k^M)$

General Minimum Change Problem

Problem: You want to give M cents in as few coins as possible using coins worth the following amounts: $C = c_1, c_2, \ldots, c_k$. Assume you can give exact change.

- In general, greedy approach won't work (see prior example)
- Solving via recursive formulation is $O(k^M)$

```
MinChange(M):
    if M = 0:                    <-- base case
        return 0
    opt = []
    for i = 1 to k:
        if c_k ≤ M:
            opt.append(1 + MinChange(M-c_k))   <-- recursive case
    return min(opt)
```

Recursion Tree

- The direct recursion approach is really awful!
- Exponential time in the worst case
- **However**: there are only `M` unique subproblems (why?)

Recursion Tree

- The direct recursion approach is really awful!
- Exponential time in the worst case
- **However**: there are only `M` unique subproblems (why?)
- We can save work by solving each subproblem once and storing the value

Step 2: Recurrence Relation

```
MinChange(M):
    if M = 0:
        return 0
    opt = []
    for i = 1 to k:
        if c_k ≤ M:
            opt.append(1 + MinChange(M-c_k))
    return min(opt)
```

- Can represent optimal solution via a recurrence relation

$$OPT(M) = \begin{cases} 0 & \text{if } M = 0 \\ 1 + min_{c \in C, c \leq M} OPT(M - c) & \text{if } M > 0 \end{cases}$$

Step 2: Recurrence Relation

```
MinChange(M):
    if M = 0:
        return 0
    opt = []
    for i = 1 to k:
        if c_k ≤ M:
            opt.append(1 + MinChange(M-c_k))
    return min(opt)
```

Tip: start writing the recursive algorithm and translate it into a recurrence

- Can represent optimal solution via a recurrence relation

$$OPT(M) = \begin{cases} 0 & \text{if } M = 0 \\ 1 + min_{c \in C, c \leq M} OPT(M - c) & \text{if } M > 0 \end{cases}$$

Step 3: Iterative Bottom-Up Algorithm

Idea: compute the optimal value of every unique subproblem in order from smallest (base case) to the largest (original problem) using recurrence

- We call this "inductively" computing the optimal value

```
OptChange(M):
    initialize list L of size M to hold optimal values
    L[0] = 0
    for i = 1 to M:
        opt = []
        for j = 1 to k:
            opt.append(1 + L[max(i – cⱼ, 0)]]
            L[i] = min(opt)
    return L
```

Step 3: Iterative Bottom-Up Algorithm

Idea: compute the optimal value of every unique subproblem in order from smallest (base case) to the largest (original problem) using recurrence

- We call this "inductively" computing the optimal value

```
OptChange(M):
    initialize list L of size M to hold optimal values
    L[0] = 0
    for i = 1 to M:
        opt = []
        for j = 1 to k:
            opt.append(1 + L[max(i – cⱼ, 0)]]
            L[i] = min(opt)
    return L
```

- Running time?

Step 3: Iterative Bottom-Up Algorithm

Idea: compute the optimal value of every unique subproblem in order from smallest (base case) to the largest (original problem) using recurrence

- We call this "inductively" computing the optimal value

```
OptChange(M):
    initialize list L of size M to hold optimal values
    L[0] = 0
    for i = 1 to M:
        opt = []
        for j = 1 to k:
            opt.append(1 + L[max(i – cⱼ, 0)]]
            L[i] = min(opt)
    return L
```

- Running time? `O(Mk)`

Step 3: Iterative Bottom-Up Algorithm

Do we need to compute the optimal subproblem for every value between 1 and M?

Step 3: Iterative Bottom-Up Algorithm

Do we need to compute the optimal subproblem for every value between 1 and M?

- No, only for values our coins can produce

Step 3: Iterative Bottom-Up Algorithm

Do we need to compute the optimal subproblem for every value between 1 and M?

- No, only for values our coins can produce

Example: suppose we have coins with values 5, 3

- We need to check subproblems 0, 3, 5, 6, 8, 9, 10, 11, 12, 13, 15, …
- But not subproblems 1, 2, 4, 7, 14, …

Step 3: Iterative Bottom-Up Algorithm

Do we need to compute the optimal subproblem for every value between 1 and M?

- No, only for values our coins can produce

Example: suppose we have coins with values 5, 3

- We need to check subproblems 0, 3, 5, 6, 8, 9, 10, 11, 12, 13, 15, …
- But not subproblems 1, 2, 4, 7, 14, …
- In general, we need to check any subproblem `J` such that `J = 3x + 5y`, where `x, y ≥ 0`

Step 3a: Top-Down Memoization Algorithm

- An alternate approach to bottom-up iteration
- Keep the recursive function structure, but store value in array on first computation and reuse it
- Can be useful if the subproblems we need to check are sparse
- Typically, same running time as Iterative Bottom-Up approach
- **In practice**: memoization is slower and uses more memory due to function call overhead and stack management

Step 4: Recover the Solution

Idea: save the best choice from each subproblem and backtrack

Step 4: Recover the Solution

Idea: save the best choice from each subproblem and backtrack

First, modify `OptChange` to track the choice at each step

```
OptChange(M):
    initialize list L of size M to hold optimal values
    initialize list B of size M to hold optimal choices
    L[0] = 0
    B[0] = 0
    for i = 1 to M:
        opt = []
        for j = 1 to k:
            opt.append(1 + L[max(i – cⱼ, 0)]]
            L[i] = min(opt)
            B[i] = which min(opt)
    return L
```

returns index of min

Step 4: Recover the Solution

Idea: save the best choice from each subproblem and backtrack

Then, find out which choices were made

```
getSolution(M):
    given lists L, B
    A = {}
    i = M
    while i > 0:
        j = B[i]
        A.add(j)
        i = i - c_j
    return A
```

Step 4: Recover the Solution

Idea: save the best choice from each subproblem and backtrack

Then, find out which choices were made

```
getSolution(M):
    given lists L, B
    A = {}
    i = M
    while i > 0:
        j = B[i]
        A.add(j)
        i = i - c_j
    return A
```

- Running time?

Step 4: Recover the Solution

Idea: save the best choice from each subproblem and backtrack

Then, find out which choices were made

```
getSolution(M):
    given lists L, B
    A = {}
    i = M
    while i > 0:
        j = B[i]
        A.add(j)
        i = i - c_j
    return A
```

- Running time? `O(M)`

Recap

Recursive algorithm -> find recurrence -> write iterative algorithm

Three ways of expressive value of optimal solutions to subproblems

- MinChange(M): recursive algorithm
- OPT(M): recurrence relation
- L: array holding optimal value at each distinct subproblem

Next Time

- Steel cutting problem